Realtime Caption & Translation Overlay App

Technical Specification, Product Requirements, and AI Coding Agent Plan

Audience: Engineers, product stakeholders, UX/design, QA, and AI coding agents

Primary goal: local-first real-time subtitles and translation with excellent UX and minimal system resource usage

Target platforms: Windows, macOS, Linux first; Android/iOS later through shared speech core

Product principle: If the app feels slow, clunky, ugly, or confusing, it fails - even if the ASR works.

1. Executive Summary

Build a lightweight cross-platform desktop app that captures selected system audio or microphone audio, performs local automatic speech recognition, optionally translates the recognized text, and displays it in a movable translucent subtitle overlay. The product must feel instant, clean, and effortless. The technical design must prioritize low CPU/RAM usage, privacy, and maintainability.

Decision	Recommendation	Reason
App shell	Tauri 2 + Rust core + thin TypeScript UI	Smaller and lighter than Electron; Rust handles performance-critical work.
Frontend	Svelte or Solid preferred; React acceptable if team speed is higher	Overlay UI is simple; avoid unnecessary runtime and re-render overhead.
ASR engine	whisper.cpp as default engine	Local, cross-platform, quantized, CPU-first, mobile-compatible roadmap.
VAD	Silero VAD before ASR	Skip silence and reduce inference load.
Storage	SQLite	Single-file, local, queryable transcripts and settings.
Translation	Phase 1: Whisper translate-to-English; Phase 2: local text translation	Start simple, then support language-to-language translation.
UX priority	Overlay responsiveness and readability above feature count	The app must feel invisible until needed.

Product values

- UX first: the user should understand the app within 30 seconds.
- Lightweight by default: low idle CPU, bounded memory, no bloated runtime features.
- Local-first privacy: audio should stay on-device unless the user explicitly enables a cloud option later.
- Fast feedback: captions should appear quickly, with stable text replacing provisional text.
- Cross-platform discipline: shared Rust core, platform-specific capture adapters.
- Accessibility by design: high contrast, readable fonts, keyboard shortcuts, predictable overlay behavior.

2. Product Scope

MVP desktop scope

- Dashboard window with source selection, model selection, language settings, and start/stop controls.
- Audio source capture: full system audio, single app where supported, microphone, and optional mixed capture.
- Translucent subtitle overlay window: always-on-top, movable, resizable, click-through toggle, opacity controls.
- Live captions: same-language transcription using local ASR.
- Translation mode: translate recognized text to selected output language where supported.
- Transcript history: save sessions locally with timestamps and export to TXT, SRT, VTT, and JSON.
- Performance presets: Low Resource, Balanced, Accuracy, Power User.

Post-MVP desktop scope

- Speaker/source labels for multi-source capture.
- Speaker-turn diarization as an optional high-resource mode.
- Custom vocabulary and correction dictionary.

- Global hotkeys and tray menu.
- Model manager with download, verify, delete, and disk usage controls.
- Optional cloud translation/ASR fallback behind explicit consent.

Mobile later

- Android/iOS should reuse the shared speech core but have different UX from desktop.
- Mobile MVP should focus on microphone conversation translation, not universal app-audio overlay capture.
- Conversation mode: split screen, one half upside down for the other speaker, one half normal for the phone owner.
- Android may later support playback capture for eligible apps; iOS is more restrictive and should be treated as a companion/adapted workflow.

3. Target Users and User Stories

User	Situation	Success Condition
Deaf / hard-of-hearing user	Joins Discord, Teams, Zoom, class, or work call.	Readable captions appear reliably without depending on the meeting app.
Noisy environment user	Is in a cafe, train, airport, open office, or crowded room.	Can follow speech without raising volume or asking people to repeat.
Movie / media viewer	Watches a film or video in another language.	Targets browser/VLC/system audio and sees translated subtitles.
Student / professional	Needs transcript after a lecture, interview, or meeting.	Exports clean transcript with timestamps.
Traveler / mobile user	Needs to speak with someone in another language.	Uses split-screen two-way translation with minimal taps.

Critical UX rules

- Never make source selection feel technical. Use app names/icons and simple labels like Chrome, Discord, System Audio, Microphone.
- The overlay must be easy to move, resize, hide, lock, and reset.
- Default overlay must look good immediately: readable font, high contrast, subtle background, sensible size.
- Settings should have presets first, advanced controls second.
- Every long-running action needs visible status: Listening, Detecting speech, Transcribing, Translating, Paused, Error.
- Error messages must say what happened and what to do next, not expose raw backend errors.

4. Core UX Flows

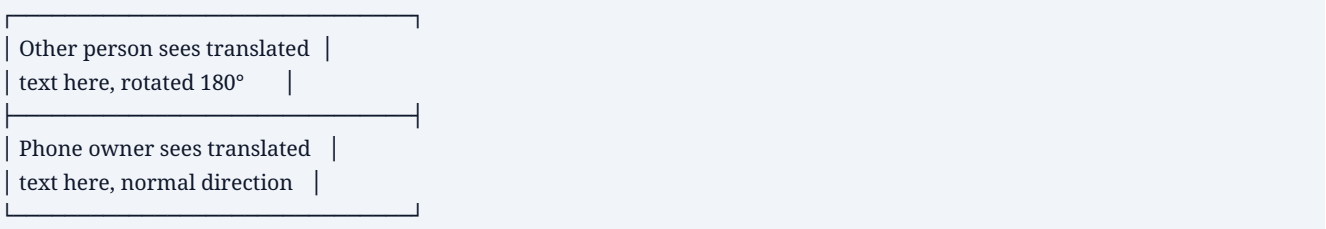
Desktop flow: live captions

1. Open app. Dashboard appears with a primary "Start Captions" path.
2. Select source: System Audio, Microphone, or Application Audio where supported.
3. Select mode: Captions only, Translate, or Original + Translation.
4. Choose preset: Low Resource, Balanced, Accuracy, or Custom.
5. Click Start. Overlay appears with short status text: "Listening...".
6. Speech is shown as provisional text, then committed into stable captions.
7. Transcript is saved locally when session ends, with export options.

Desktop flow: translated movie/browser captions

8. User selects Chrome, Firefox, VLC, or System Audio.
9. Input language is Auto or manually selected.
10. Output language is selected, e.g. English or Bosnian.
11. Overlay displays translated subtitles. Optional setting shows original line above translated line.
12. Latency target can be relaxed for better quality because movie subtitles tolerate a small delay.

Mobile flow: two-way conversation translation



- Default mode should be tap-to-speak per side because it is more reliable and lighter than automatic speaker detection.
- Hands-free mode can come later after VAD and turn-taking are stable.
- Optional text-to-speech playback can come later, but text-first is safer for MVP.

5. Functional Requirements

ID	Requirement	Priority
FR-01	List available capture sources with friendly names and icons where possible.	MVP
FR-02	Start, pause, resume, and stop live caption session.	MVP
FR-03	Create separate translucent overlay window with always-on-top support.	MVP
FR-04	Allow overlay customization: opacity, width, height, font, font size, text color, background opacity.	MVP
FR-05	Run local ASR on captured audio and display provisional + committed captions.	MVP
FR-06	Save transcript locally with timestamps and source metadata.	MVP
FR-07	Export transcript as TXT, SRT, VTT, and JSON.	MVP
FR-08	Translate ASR output to selected language where supported.	MVP+
FR-09	Show original + translation as optional dual-line subtitles.	MVP+
FR-10	Support performance presets and model selection.	MVP
FR-11	Add speaker/source labels when multiple sources are captured.	Phase 2
FR-12	Add optional diarization for mixed audio.	Phase 3
FR-13	Mobile split-screen conversation	Mobile Phase 1

	translation.	

6. Non-Functional Requirements

Category	Requirement
Performance	Idle CPU should be near-zero when not captioning. VAD should gate ASR work during silence.
Memory	Default model and buffers must fit comfortably on low-end systems. All queues must be bounded.
Latency	Calls: aim for fast provisional captions. Media translation: allow slightly higher latency for better quality.
Privacy	No audio leaves device by default. Cloud features, if added, must be opt-in and clearly labeled.
Reliability	Audio callback must never block. Capture failure must not crash the UI.
Security	Least-privilege permissions, secure auto-update, model checksum verification, no hidden recording.
Accessibility	High contrast mode, scalable fonts, keyboard shortcuts, screen-reader-friendly dashboard.
Maintainability	Platform capture adapters behind a shared interface. No duplicated business logic in frontend.

7. Recommended Tech Stack

Layer	Recommended choice	Notes
Desktop shell	Tauri 2	Best fit for lightweight cross-platform desktop and later mobile experiments.
Core language	Rust	Audio, buffering, ASR orchestration, model management, persistence, exports.
UI language	TypeScript	Thin UI only. Do not put streaming audio logic in the frontend.
Frontend framework	Svelte or Solid preferred; React acceptable	Pick based on team speed. Keep overlay simple regardless.
ASR	whisper.cpp	Default local engine with quantized models and broad platform support.
VAD	Silero VAD or equivalent ONNX/Rust-friendly VAD	Use before ASR to reduce load.
Translation	Whisper translate-to-English first; later Bergamot/Marian or NLLB distilled	Local-first strategy. Cloud optional later.
Storage	SQLite	Sessions, transcript segments, settings, export metadata.
Packaging	Tauri bundler: NSIS/MSI, DMG/App bundle, AppImage/deb/rpm	Avoid Flatpak as first Linux target due audio permission complexity.

Assessment of lightning-p2p stack

The referenced lightning-p2p direction is good: Rust + Tauri + TypeScript is aligned with this app. Reuse the philosophy, not necessarily the whole dependency set. For this app, remove anything not needed for captions, avoid animation-heavy UI dependencies, and move all performance-critical work to Rust.

Keep	Change / Avoid
Tauri 2, Rust, TypeScript, Vite	Avoid Electron unless Tauri blocks a critical OS capability.
Small native backend with clear commands/events	Avoid JS audio processing for real-time capture.
Simple desktop packaging approach	Avoid heavy animation libraries in MVP overlay.
Mobile-aware project structure	Do not force mobile UX to copy desktop UX.

8. Architecture

```
Tauri Desktop App

Main Dashboard Window              Subtitle Overlay Window
- source picker                    - translucent always-on-top view
- mode/settings                    - provisional + committed captions
- model manager                    - resize/move/lock/hide controls
- transcript viewer
        |                          ▲
        ▼                          |
Rust Core: App State + Session Manager + Event Bus
        |
        ├── Capture Adapters
        |   ├── Windows: WASAPI loopback / application loopback
        |   ├── macOS: ScreenCaptureKit
        |   └── Linux: PipeWire first, PulseAudio fallback
        |
        ├── Audio Pipeline
        |   ├── resample/downmix/normalize
        |   ├── VAD
        |   ├── rolling chunk buffer
        |   └── bounded queues
        |
        ├── Model Runtime
        |   ├── ASR engine
        |   ├── translation engine
        |   └── optional diarization
        |
        └── Persistence
            ├── SQLite settings and sessions
            └── transcript exports
```

Runtime pipeline

```
Audio Source
  ↓
Platform Capture Adapter
  ↓
PCM normalization: mono/stereo policy, sample-rate conversion, timestamps
  ↓
```

```
VAD: skip silence and music-only intervals where possible
  ↓
Chunker: rolling context window with overlap
  ↓
ASR: local speech recognition
  ↓
Caption Stabilizer: provisional text -> committed text
  ↓
Optional Translation
  ↓
Overlay + Transcript Store + Export
```

Threading and event model

- Capture callback/thread: highest priority, never blocks, writes PCM frames to bounded ring buffer.
- Audio processing worker: resampling, downmixing, timestamping, VAD.
- ASR worker: one worker by default. Additional workers only in Power User mode.
- Translation worker: separate from ASR so translation does not block transcription.
- UI event bridge: frontend subscribes to high-level events, not raw audio.
- Backpressure: if queues fill, degrade gracefully by dropping oldest provisional audio, not by freezing the UI.

9. Platform Capture Strategy

Platform	MVP capture target	Notes / limitations
Windows	WASAPI loopback for system audio; application loopback where available.	Best support for full system and app-specific capture. Start here for deepest feature completeness.
macOS	ScreenCaptureKit for app/display audio capture.	Requires permissions. Window-level visual capture does not always mean window-level audio isolation.
Linux	PipeWire first; PulseAudio monitor fallback.	Distro variability is real. App-specific capture depends on audio server/session manager behavior.
Android later	Microphone conversation mode first; optional AudioPlaybackCapture later.	Playback capture is permission-gated and app-policy-gated.
iOS later	Microphone conversation mode first.	Universal app-audio overlay is not realistic as a primary feature.

10. ASR, Translation, and Diarization Strategy

ASR model presets

Preset	Model	Target user
Low Resource	tiny / tiny.en quantized	Very weak CPUs, fallback only.
Balanced default	base / base.en quantized	Default MVP preset.
Accuracy	small / small.en quantized	Better quality on decent laptops/desktops.
Power User	larger/turbo model if available	Strong machines; not default.

Translation modes

Mode	Behavior	Phase
Off	Show original transcription only.	MVP
Translate to English	Use multilingual Whisper translation path when input is non-English.	MVP+
Original + Translation	Show two-line subtitles.	MVP+
Any language -> any language	ASR to text, then local text translation model.	Phase 2
Cloud fallback	Optional higher-quality translation through cloud API.	Later, opt-in only

Diarization

- Do not make diarization part of the MVP critical path.
- Use source/app labels first because they are cheaper and more reliable when capture sources are distinct.
- Add speaker-turn detection later for mixed streams.
- Full named-speaker diarization should be opt-in and clearly marked as higher resource usage.

11. Data Model

Entity	Key fields
settings	key, value, updated_at
sessions	id, started_at, ended_at, mode, source_summary, model_id, language_input, language_output
segments	id, session_id, start_ms, end_ms, original_text, translated_text, source_label, speaker_label, confidence, is_final
models	id, name, type, local_path, size_bytes, checksum, installed_at, last_used_at
exports	id, session_id, format, path, created_at

Single source of truth

- Rust core owns session state, capture state, model state, and transcript state.
- Frontend owns only presentation state: open panels, selected tabs, temporary form values.
- Settings are written through one SettingsService only. No direct ad-hoc config file writes from UI components.
- Transcript segments are committed once in the core and then emitted to UI. Do not duplicate transcript logic in frontend.

12. Engineering Standards

Coding practices

- Prefer explicit services over global state. Use singletons only for true process-wide resources such as AppState, SettingsService, ModelRegistry, and EventBus.
- All platform behavior must be behind traits/interfaces: CaptureAdapter, AsrEngine, TranslationEngine, TranscriptStore.

- No duplicate business logic between frontend and backend. Frontend calls commands and renders returned state.
- Use bounded channels/ring buffers for audio data. Never allow unbounded memory growth.
- No blocking work in audio callbacks. No network calls or DB writes in capture callbacks.
- Use typed events and typed command responses. Avoid stringly-typed IPC payloads.
- Validate every external path, model file, and export destination.
- Use structured logging with redaction. Never log raw audio. Avoid logging full transcripts unless debug mode explicitly enables it.
- Fail gracefully: stop session, release capture device, preserve transcript so far, show user-safe error.

Security and privacy

- Default behavior: fully local processing. No account and no cloud required.
- User must explicitly start capture. Show persistent visible status while listening.
- Require explicit permission for microphone/system capture where OS requires it.
- Model downloads must use HTTPS and checksum verification.
- Auto-update must be signed. Do not execute unsigned downloaded binaries.
- Store transcripts locally. Give user clear delete/export controls.
- For future cloud features, require opt-in per session and disclose what is sent.

13. Product Backlog Items and Iterative Coding Agent Strategy

The AI coding agent should not attempt to build the full app in one pass. Work in thin vertical slices. Each PBI must compile, run, and have acceptance tests before the next PBI begins.

PBI	Goal	Acceptance criteria
PBI-00	Create base repository and Tauri app skeleton.	App launches on dev machine. Main window shows placeholder dashboard. CI runs lint/build.
PBI-01	Define Rust core architecture and typed IPC.	Traits/services exist. Frontend can call health/status command. No real audio yet.
PBI-02	Create overlay window.	Button opens translucent always-on-top overlay. Overlay text can be updated through event.
PBI-03	Settings single source of truth.	Overlay opacity, font size, and text color persist through SQLite/settings service.
PBI-04	Audio capture adapter scaffold.	Mock capture adapter emits test audio/timestamps. Real adapters behind same interface.
PBI-05	Windows system audio capture MVP.	Can capture system audio on Windows and show input level meter. No ASR yet.
PBI-06	Audio pipeline: resample, downmix, VAD.	Silence is detected. Logs show speech segments. UI remains responsive.
PBI-07	ASR engine integration.	Captured or test audio becomes live provisional captions in overlay.
PBI-08	Caption stabilizer.	Text transitions from provisional to committed. No flickering wall of changing text.
PBI-09	Transcript persistence and export.	Session saves locally. Export

		TXT/SRT/VTT/JSON works.
PBI-10	Translation MVP.	User can select translate-to-English mode. Overlay shows translated text.
PBI-11	macOS capture adapter.	App lists and captures supported macOS sources with permission guidance.
PBI-12	Linux capture adapter.	PipeWire capture works on supported Linux; PulseAudio fallback documented.
PBI-13	Polish and UX QA.	First-run flow, empty states, errors, hotkeys, reset overlay, visual polish.
PBI-14	Speaker/source labels.	Distinct captured sources can label transcript segments.
PBI-15	Mobile proof-of-concept.	Shared Rust core used in simple mobile conversation translation prototype.

Agent workflow rules

13. Before coding each PBI, inspect existing files and summarize the current architecture in 5 bullets.
14. Implement the smallest vertical slice that satisfies the PBI.
15. Do not introduce a new dependency without explaining why it is needed and why a smaller alternative is not enough.
16. After changes, run format, lint, typecheck, unit tests, and a build where practical.
17. Report changed files, what was implemented, how to test manually, and known limitations.
18. Never rewrite large unrelated files just to change style. Keep diffs focused.
19. Do not duplicate logic across OS adapters. Put shared code in the Rust core.

14. Testing Strategy

Test type	What to test
Unit tests	Chunking, timestamp math, caption stabilization, export formatting, settings validation.
Integration tests	Mock capture -> VAD -> ASR mock -> overlay event -> transcript save.
Platform tests	Windows WASAPI, macOS ScreenCaptureKit permission flow, Linux PipeWire/PulseAudio behavior.
Performance tests	Idle CPU, active CPU, memory by model preset, latency from audio input to visible caption.
UX tests	First-run clarity, overlay readability, hotkeys, reset behavior, error recovery.
Security tests	Path validation, model checksum, update signing, no hidden capture, transcript deletion.
Accessibility tests	Keyboard navigation, color contrast, scalable fonts, screen-reader labels on dashboard.

Definition of Done

- Feature works on at least the current target OS for that PBI.
- UI does not freeze during capture or inference.
- Memory use is bounded and does not grow during a 30-minute session.

- Errors are user-readable and logged for debugging.
- Settings and transcript state use the defined services only.
- Manual test steps are documented in the PR or agent response.

15. AI Coding Agent Master Prompt

Use this prompt as context when instructing the coding agent. Paste it together with this PDF or the relevant section of the spec.

```
You are building a lightweight cross-platform desktop app for live subtitles and translation.

Core values:
- UX is the top priority. The app must feel simple, fast, readable, and reliable.
- Speed and efficiency matter. Keep CPU, RAM, startup time, and bundle size low.
- Local-first privacy. Audio stays on-device by default.
- Maintainability matters. Use single source of truth, typed interfaces, and no duplicated business logic.

Target architecture:
- Tauri 2 desktop app.
- Rust core owns capture, audio pipeline, ASR, translation, settings, transcript storage, and exports.
- TypeScript frontend is thin and only renders state / sends commands.
- Use platform-specific CaptureAdapter implementations behind a common Rust trait.
- Use whisper.cpp for ASR, VAD before ASR, and SQLite for local persistence.

Development strategy:
- Work one PBI at a time.
- Before coding, inspect the repo and summarize the relevant current structure.
- Make the smallest useful vertical slice.
- Avoid new dependencies unless clearly justified.
- No blocking work in audio callbacks.
- Use bounded queues for audio and inference pipelines.
- Do not put streaming audio logic in the frontend.
- Do not duplicate settings, transcript, or caption logic between frontend and backend.
- After implementation, run format/lint/tests/build where possible and report exact manual test steps.

For every task, return:
1. What you changed.
2. Files changed.
3. How to run/test it.
4. Any tradeoffs or limitations.
5. What PBI should come next.
```

16. Recommended MVP Build Order

Sprint	Deliverable	Why this order
Sprint 1	Base app, overlay window, settings persistence, mock captions.	Proves UX foundation before hard ASR work.
Sprint 2	Windows system audio capture + level meter + VAD.	Proves capture pipeline and resource behavior.
Sprint 3	ASR integration with local model and caption stabilizer.	Creates first real product loop.
Sprint 4	Transcript storage/export and model	Makes sessions useful after live

	presets.	captioning.
Sprint 5	Translation MVP and media subtitle polish.	Creates strongest marketable differentiator.
Sprint 6	macOS/Linux adapters and platform QA.	Expands cross-platform support after core is stable.
Sprint 7	Speaker/source labels and advanced settings.	Adds power features without bloating the MVP.
Sprint 8	Mobile conversation translation prototype.	Reuses speech core with adapted mobile UX.

17. Risks and Mitigations

Risk	Mitigation
ASR latency feels too slow.	Use provisional captions, VAD, smaller default model, and model presets.
Overlay feels annoying.	Add lock, hide, reset, opacity presets, click-through toggle, and keyboard shortcuts.
App-specific capture differs by OS.	Expose OS capability honestly and build platform adapters.
Translation quality varies.	Start with clear language support list and show original + translation option.
Resource use grows over long sessions.	Bound all queues, stream transcript writes, run long-session tests.
Coding agent creates duplicated architecture.	Use traits/services, single source of truth, PBI acceptance criteria, and focused diffs.

18. Source Notes

Research basis used for this specification includes official and primary documentation for Tauri, Microsoft WASAPI loopback/application loopback, Apple ScreenCaptureKit, PipeWire/PulseAudio, whisper.cpp, OpenAI Whisper model details, Silero VAD, SQLite, Android AudioPlaybackCapture, Apple ReplayKit, sherpa-onnx, faster-whisper, Vosk, pyannote.audio, and the referenced lightning-p2p repository. The engineering recommendation is based on combining those constraints with the product goal: best UX with minimal resource use.